

HPPS Assignment A3

Björn Petterson (zts900), Manar(chs380), Thomas Andersen (kgq325)

April 15, 2026

Introduction

One of the key challenges we have faced in working with this assignment has been figuring out how to work with indexes instead of the data itself. However a meta task has been working with a code base that we did not completely write ourselves. Getting a sense of the interactions between files and functions has been quite the challenge.

One takeaway from the week and this assignment has been the front-loading of computationally heavy tasks as illustrated by constructing the `kdtree` before you do the actual queries. This will not work for every scenario or program, but it is clear that being aware of what computation can be done ahead of time can lead to powerful performance gains.

Note regarding compilation: Running `make` without arguments compiles all, and should do so without warnings or errors.

util.c & io.c

In `util.c` we implemented the functions `distance()` and `insert_if_closer()`. These functions were used in order to calculate the Euclidean distance between two points and to update the nearest point by comparing with the already nearest neighbor, respectively. In `io.c` we have various functions to handle the input and output operations for indexes. `read_points()` reads a set of `n` points, each with `d` dimensions from a file, and allocates memory to store the points.

`read_indexes()` reads an array of indexes from a file, which is useful to compare the expected and real results.

`write_points()` write points writes `n` points, each with `d` dimensions, to a file `write_indexes()` writes the nearest neighbor indexes for `n` queries with `k` neighbors to a file.

Task 1.1: Brute-force k-NN

The functionality is correct, we can see that the Brute-force k-NN iterates successfully through all reference points in order to maintain a sorted list of the k closest points to the query.

We can always verify its functionality by testing it and see whether our program returns the expected output.

We could point out that the time complexity of $O(n)$ per query is okay while we aren't working with large dataset, which is why we implement the k-d tree which reduce the search space for k-NN queries.

In order to verify the correctness visually, for small dataset, we used `knn-genpoints` command to generate a dataset of points and queries. We then ran `knn-bruteforce()` to compute the k-nearest neighbors. Finally we used the `knn-svg()` command to create the visual representation and open the saved .svg file.

Task 1.2: k-NN with a k-d tree

kdtree construction

The way we decided to handle the interaction between the creation of nodes and trees was to let `kdtree_create` call `kdtree_create_node()` which recursively creates nodes until all points in the relevant file have been created.

When `kdtree_create_node()` is called it will create an index and sort the index. With the help of ChatGPT we have created a function `cmp_indexes()` to replace `cmp_string()` that we could use in conjunction with `hpps_quicksort()`. It compares points by axis, so that we get sorted indexes. We use a `struct` to maintain context for axis calculation to index into the points being compared and locate the right values for comparison.

kdtree traversal

Following the described pseudo code steps we implemented `kdtree_knn_node` that constructs the array `closest` containing the indexes of the k closest neighbors for each query point.

We believe the function is functionally correct but could be optimized. We calculate

the distance to the furthest point through a loop, but if we instead could insert into `closest` in a way that kept it sorted this might be minimized. Also if we could get around calculating the number of elements though a loop by just passing in the number of elements each recursive call this could also be optimized. The reason it matters is that we potentially make $\log(n)$ recursive calls so what happens in that call is a factor of n important. We don't allocate memory in the function since we don't have any temporary arrays, so no memory leaks.

Task 3: Testing and benchmarking

Memory leaks

We have tested the following implementation files for memory leaks by running `clang` with the `-fsanitize=address` flag which got the following result:

- `io.c / util.c / kdtree.c`: No reported leaks (malloc: nano zone abandoned)

Visual test small dataset

We observe that we get identical images for `brute-force` and `kd-tree`. This test was run on 20 points with 3 neighbors for 2 queries. We also observe that the number neighbors within the 2 circles are in fact 3.

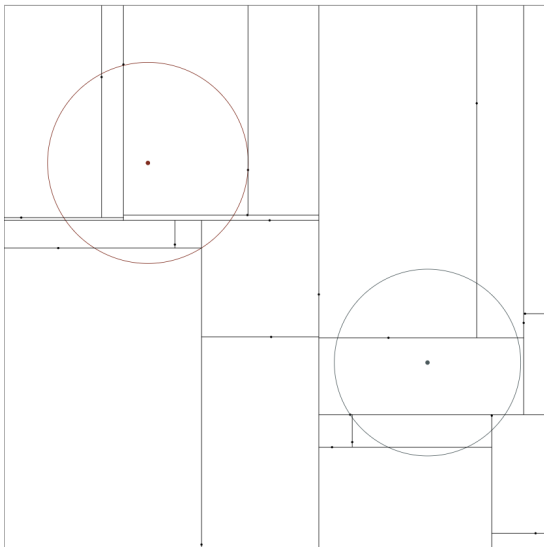


Figure 1: Brute-force result.

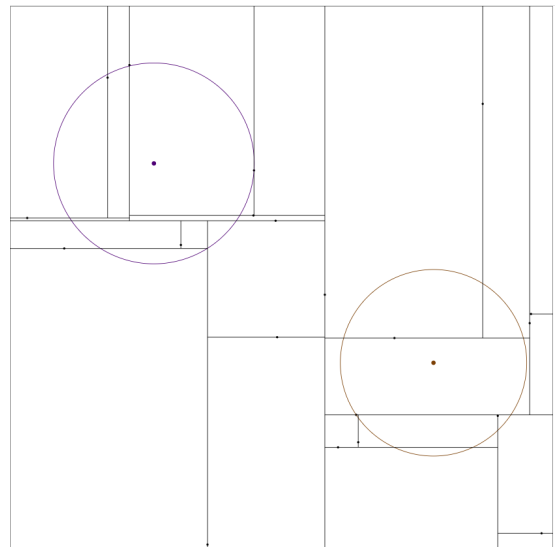


Figure 2: k-d tree result.

Large dataset, check indexes

A program `knn-verify-large.c` was written to check that no point referenced by indexes here the k neighbors for a given query point were actually the k closest points. We calculate the distance to the furthest away neighbor and the minimum distance to a none neighbor and compare. We wrote the pseudo code but had an LLM write most implementation since we didn't have time to fully implement this ourselves. Running the test on a dataset of 10,000 points and 1,000 queries and the resulting indexes from `bruteforce` and `kd-tree` showed no closer points than those in neighbor for either algorithm.

Compare results brute force vs kdtree

The testing program `knn-verify.c` was written to take in two index files written by `bruteforce` and `knn-kdtree` for the same input points and queries. The test loops through the number of queries and tests that the provided indexes for each query are the same. We got some LLM help here although the pseudo code and design, debugging were all human. This test showed for 1,000 points 2 queries 100 neighbors no difference.

We have not included the the testing files above in the final package as they were created purely for testing purposes.

Time comparison

Two tests were created, one small test with 100 points and 10 queries, and one large test with 100,000 points and 1,000 queries. Both tests used 2 dimensions. The brute force algorithm and `kd-tree` algorithm were then compared. Unsurprisingly both were very fast for the small dataset here the brute force were even faster in real time and user time. However the interesting difference was for the large dataset where for user time we saw `bruteforce` taking 6.8 seconds vs only 0.6 seconds for `kd-tree`. A quite impressive difference. See table1.

Table 1: Runtime Comparison for k-NN Algorithms

Algorithm	Points	Queries	Real Time (s)	User Time (s)	System Time (s)
Brute Force	100	10	0.019	0.002	0.001
Brute Force	100,000	1,000	7.369	6.841	0.019
K-d Tree	100	10	0.020	0.004	0.000
K-d Tree	100,000	1,000	0.709	0.558	0.029